# CIVICVOICE: A Transformer-Based Framework for Multi-Channel Citizen Feedback Analysis in Public Services

Peddi Reddy Tejaswini, Inchara K. R., Sahana Haragapure

School of AI and Advanced Computing, Presidency University, Bengaluru, Karnataka, India

**Abstract—**Public-service organizations receive citizen feedback through multiple channels, including online grievance portals, social media, email, surveys, and local complaint systems. The information can reveal service-delivery problems, recurring concerns, and urgent cases, but its volume and unstructured nature make manual review difficult. CIVICVOICE is a proposed AI-assisted framework for organizing and analysing multi-channel citizen feedback. The project combines a consolidated feedback dataset, text preprocessing, transformer-oriented natural-language processing (NLP), a relational database, a backend API, and a dashboard intended to help users explore feedback and identify patterns. The project dataset has been reported to contain approximately 205,168 records across seven columns. Previous preprocessing summaries report that most records were retained and that cleaned text was produced for nearly all records; these figures should be reproduced from the current pipeline before formal publication. The dashboard prototype presents high-level indicators, source and location summaries, trends, and NLP insight areas. This paper documents the motivation, system design, data workflow, implementation status, and evaluation plan. It distinguishes implemented components from prototype functionality and does not claim model accuracy or operational impact where reproducible measurements are not yet available. The main contribution is an integrated project architecture for turning heterogeneous citizen messages into a structured workflow for exploration and future transformer-based analysis. Further work will validate the selected transformer model, create a labelled evaluation set, measure classification performance, complete backend–database integration, and assess privacy, fairness, and usability.

**Index Terms—**Citizen feedback analysis, public services, natural language processing, transformer models, text analytics, FastAPI, MySQL, dashboard, grievance analysis.

## I. INTRODUCTION

Citizen feedback is an important source of evidence about the performance of public services. Residents may report delayed services, infrastructure problems, sanitation concerns, administrative difficulties, public safety issues, or dissatisfaction with the handling of a request. These reports arrive in different formats and through different channels. Some are short and direct, while others are long descriptions containing multiple issues, local names, abbreviations, spelling variations, or contextual details. A public-service team that reviews each message manually may find it difficult to identify recurring concerns quickly, compare patterns across locations, or prioritise reports consistently.

Digital platforms have increased the quantity and variety of feedback available to public agencies. However, collecting messages is only the first step. To make the information useful, a system must preserve the original content, standardise records where possible, support searching and filtering, and provide analytical views that help users understand the data. Text analytics can assist with grouping, classifying, and summarising messages, but its outputs must be treated as decision support rather than as unquestionable judgements.

CIVICVOICE—an AI assistant for multi-channel citizen feedback analysis—is being developed to connect data preparation, natural-language processing, data storage, backend services, and dashboard-based exploration. The project brings together feedback described as originating from sources such as CPGRAMS, Bengaluru-local complaints, surveys, Twitter, and email. The current project summary reports a consolidated dataset of approximately 205,168 records and seven columns. The exact source coverage, record counts, and field definitions should be confirmed against the final dataset version used for deployment.

The NLP workflow includes text cleaning and a transformer-based direction for semantic analysis. Unlike approaches that represent each document only through independent word counts, transformer language models can use contextual relationships between tokens to form representations that depend on surrounding text. This can be useful for citizen messages where the same word may convey different meanings in different contexts. Nevertheless, a transformer model does not automatically guarantee better results: performance depends on the selected model, training data, task definition, class balance, and evaluation method.

The project also includes a MySQL database design and a FastAPI backend setup. The dashboard prototype is intended to expose summary metrics, feedback trends, an explorer for individual records, and an area for AI/NLP insights. These components form the path from raw text to a user-facing analysis workflow. At the time of this draft, some dashboard insight cards have been identified as prototype elements, and a successful API health response alone does not establish that all database operations and NLP endpoints are integrated.

The objectives of this paper are: (1) to describe the need for a multi-channel citizen-feedback analysis system; (2) to document the data and preprocessing workflow; (3) to present the planned transformer-based NLP role without inventing an unverified model configuration; (4) to describe the database, backend, and dashboard architecture; (5) to define an evaluation plan that can produce defensible results; and (6) to discuss limitations and responsible-use requirements.

## II. RELATED WORK AND TECHNICAL BACKGROUND

Citizen feedback analysis sits at the intersection of public-sector digital services, text mining, information retrieval, and machine learning. Grievance portals and service-request systems help agencies collect complaints in a structured manner, but free-text descriptions remain difficult to analyse at scale. A common analytical objective is to convert unstructured text into categories or indicators that can be reviewed by staff. Other objectives include detecting repeated topics, examining changes over time, comparing geographical areas, and identifying messages that may need faster human attention.

Traditional text-processing pipelines often begin with tokenisation, normalisation, stop-word handling, and vectorisation. Methods such as term frequency–inverse document frequency (TF-IDF) can provide useful sparse representations for search or classification baselines. These methods are computationally efficient and relatively easy to interpret, but their representations do not inherently model the full contextual meaning of a sentence. A baseline remains valuable even when a project intends to use transformers, because comparison against a simpler method helps determine whether added complexity is justified.

Transformer architectures use attention mechanisms to represent relationships among tokens. Pretrained language models learn general linguistic patterns from large text corpora and can then be adapted to tasks such as sentiment classification, topic classification, or text-pair matching. Contextual representations may help distinguish expressions that share words but differ in meaning. For example, a message saying that a road is "blocked" differs operationally from one saying that a road is "not blocked anymore," even though both contain similar terms. Transformer models can be fine-tuned using task-specific labelled examples, or used as part of other inference pipelines depending on the implementation.

The term "transformer-based analysis" describes a family of approaches rather than one fully specified model. A reproducible study must name the exact architecture and checkpoint, tokeniser, maximum sequence length, training configuration, labels, split strategy, software versions, and evaluation procedure. Those details are not assumed in this draft because the available project context does not confirm a final model name or a completed, measured evaluation. The implementation should record these details once they are fixed.

Related work in public-service analytics also raises questions beyond predictive performance. A high score on a test set does not establish that a system is fair across locations, languages, writing styles, or user groups. Nor does an urgency score establish that a complaint is objectively urgent. Systems that rank or classify feedback should expose enough context for human reviewers to inspect the original message and correct mistakes. They should also avoid presenting inferred sentiment or topic as verified fact.

The project's earlier documentation contains references to work on public-service or feedback analytics. The references listed at the end of this draft are retained as starting points from the project materials, but each should be checked against the original publication record before submission. A final literature review should add recent, directly relevant research on

citizen complaint classification, transformer-based text classification, public-sector AI governance, and multilingual or noisy-text analysis.

## III. PROBLEM STATEMENT AND OBJECTIVES

The central problem addressed by CIVICVOICE is that feedback collected through multiple channels is difficult to inspect consistently when records are fragmented, text is noisy, and users lack a common analytical interface. Staff may need to open separate systems, manually search messages, or rely on aggregate counts that do not explain the underlying text. Such a workflow can slow the identification of repeated service issues and make comparisons across sources more difficult.

The system aims to create a common pipeline in which feedback records can be consolidated, cleaned, stored, accessed through an API, and explored through a dashboard. The NLP component is intended to help interpret textual content and support analytical views. The design does not assume that every message can be assigned a single correct label, and it does not replace official grievance-handling procedures or human review.

The specific objectives are:
1. Consolidate citizen feedback from the data sources included in the project dataset while preserving source information and traceability.
2. Apply consistent text preprocessing to reduce formatting noise and prepare text for downstream NLP tasks.
3. Use a transformer-based approach for contextual language analysis once the task, model, and labels are confirmed.
4. Store feedback and associated analysis fields in a relational MySQL schema.
5. expose backend endpoints through FastAPI so that the dashboard and other clients can retrieve data through a defined interface.
6. Present summary metrics, trends, searchable feedback, and analysis outputs in a dashboard.
7. Evaluate the NLP component with a labelled test set and report both overall and class-wise metrics.
8. Document operational limitations, privacy requirements, and the need for human oversight.

## IV. SYSTEM ARCHITECTURE

CIVICVOICE can be understood as a layered system with five connected parts: data ingestion, preprocessing, NLP analysis, storage and API services, and dashboard presentation. Each layer has a separate responsibility. This separation is intended to make the project easier to test and extend, and to reduce the risk of mixing data-cleaning logic with interface code.

A. Data ingestion and consolidation. Feedback records are gathered from the sources available to the project. Source-specific fields and formats may differ, so the ingestion step should map each source to a common set of fields while retaining source identity and any available timestamp or location. The ingestion process should also record provenance and avoid silently overwriting original messages.

B. Text preprocessing. Raw text may contain repeated whitespace, markup, URLs, mentions, unusual punctuation, encoding artefacts, and inconsistent casing. Preprocessing creates a normalised text field for downstream analysis. The raw text should remain available where permitted so that reviewers can compare a model result with the original message. Cleaning must be conservative: removing too much text can erase context, negation, urgency signals, or location references.

C. Transformer-based NLP. The NLP layer receives prepared text and produces outputs defined by the chosen task. Possible tasks include sentiment classification, topic or issue categorisation, and prioritisation support. These are distinct tasks and should not be treated as interchangeable. Each output should be stored with its model/version identifier and, where available, a confidence score. The system should allow “unknown” or “needs review” outcomes rather than forcing every message into a category.

D. MySQL data layer. The relational database is intended to hold feedback records and related analysis information. A practical schema separates the core feedback record from derived predictions when this improves maintainability. Stable identifiers connect records and predictions. Fields such as source, timestamp, location, original text, cleaned text, predicted category, sentiment, urgency, confidence, and model version should be included only when they exist in the implemented pipeline and have a clearly defined meaning.

E. FastAPI backend. The backend provides an application interface between the database, NLP functions, and dashboard. Typical endpoint responsibilities include health checks, feedback retrieval, filtered search, aggregate statistics, and analysis requests. The API should validate query parameters, return predictable JSON structures, handle missing records gracefully, and avoid exposing database credentials. A health endpoint that responds successfully confirms that the service is running, but separate tests are needed for database connectivity and each endpoint.

F. Dashboard. The dashboard is the user-facing layer. Current prototype screens include a high-level overview, feedback trend views, a feedback explorer, and an AI/NLP insights area. These screens are intended to support quick scanning and drill-down into records. The displayed values should ultimately come from the backend rather than be hard-coded, and the interface should make clear when a metric is estimated, sampled, or generated by a prototype.

The overall data flow is: source records → common dataset → cleaned text → NLP inference or analysis → persisted results → FastAPI endpoints → dashboard views. In a complete implementation, this flow should be reproducible and testable end to end. The current project state includes dataset preparation, database design/setup, a backend service setup, and a dashboard prototype; the level of integration between all components should be confirmed through tests before the system is described as production-ready.

## V. DATASET DESCRIPTION AND PREPROCESSING

The project context reports a consolidated dataset of approximately 205,168 feedback records organised across seven columns. Sources described in the project include CPGRAMS, Bengaluru-local complaints, surveys, Twitter, and email. These sources are potentially heterogeneous: some records may be formal grievances, others social posts or survey responses, and some may have different levels of metadata. The dataset should therefore be treated as a project-specific collection rather than assumed to be a statistically representative sample of all citizen feedback.

A. Data fields and provenance. Before publication, the final paper should include a data dictionary showing the name, meaning, data type, and missing-value rate for each field. It should also report how records from different sources were mapped to the common schema. If source counts are included, they must be computed from the exact dataset used for the reported experiments. Earlier project summaries contain approximate source-level counts that do not exactly reconcile with the reported master total; they should not be reproduced as definitive counts without re-running the data audit.

B. Preprocessing workflow. The described workflow includes producing a cleaned-text field and inspecting the number of records retained. Typical cleaning operations may include trimming leading and trailing spaces, normalising repeated whitespace, handling null or empty strings, standardising encodings, and removing selected formatting noise. The exact operations should be listed from the final script rather than inferred. For example, URL removal may be useful for some classification tasks but harmful when a link itself identifies the service or incident. Similarly, removing punctuation indiscriminately can weaken the meaning of short messages or abbreviations.

C. Reported preprocessing summary. A prior project summary reported that approximately 205,168 records were retained after preprocessing, around 184,072 records had changes in cleaned text, and one record had empty cleaned text. It also reported an average text length changing from roughly 729 to 716 characters. These are historical project figures, not independently re-computed results for this paper. They should be treated as provisional until the latest preprocessing scripts are run on a versioned input file and the counts are regenerated. The final report should state the input filename or dataset version, date of processing, script version, and exact definition of “changed text.”

D. Data quality checks. The pipeline should measure duplicate records, missing text, malformed timestamps, missing locations, unusual text lengths, and inconsistent source labels. If records are deduplicated, the rule should be explicit. Duplicate-looking messages may represent repeated reports of the same issue, so deduplication should not remove meaningful evidence without a documented rationale. Where the same complaint is copied across channels, the system may need both a canonical record and links to the source occurrences.

E. Train, validation, and test partitions. If supervised learning is used, records must be divided into training, validation, and test sets before model selection. Near-duplicate records should not be allowed to leak across splits because they can inflate test performance. A time-based split may be

appropriate if the intended use is to analyse future feedback. If multiple sources are included, source-wise evaluation can reveal whether the model works similarly on formal grievances, social posts, and survey responses.

F. Label quality. A transformer classifier requires labels or a clearly specified alternative method. The paper should document who labelled examples, the label definitions, how ambiguous cases were handled, and whether annotators agreed. If labels are generated automatically or inherited from source metadata, that limitation should be disclosed. Sentiment, topic, and urgency are not equivalent labels; each requires its own definition and evaluation.

The dataset provides a substantial base for building an analysis workflow, but size alone does not demonstrate model quality. Reliability depends on provenance, consistency, label quality, leakage control, and reproducible evaluation.

## VI. TRANSFORMER-BASED NLP APPROACH

The project uses a transformer-oriented NLP direction rather than treating traditional bag-of-words features as the primary approach. A transformer model processes a tokenised input sequence and uses contextual attention to construct representations of the text. For a citizen-feedback system, the purpose is to capture context that may be missed by simple keyword matching, including negation, phrasing variation, and relationships between terms.

A. Task definition. The model should be selected only after the target task is defined. Sentiment analysis estimates the expressed polarity or tone of a message; topic classification assigns a message to an issue category; urgency classification estimates whether a message warrants faster review under a defined policy. These outputs answer different questions. A message can be negative but not urgent, or urgent without strongly negative sentiment. The system should not collapse all outputs into a single score unless that score has a clearly documented interpretation.

B. Tokenisation and input preparation. Transformer models generally use a model-specific tokenizer that converts text into token identifiers and attention masks. Maximum sequence length affects computational cost and how much of a long complaint is represented. Because the project's text-length summaries indicate that messages can be long, the implementation should document whether text is truncated, split into segments, or summarised before inference. Truncation may remove important details near the end of a complaint; segmentation may preserve more content but requires a rule for combining segment-level predictions.

C. Model choice and training. A final experiment should identify the exact pretrained checkpoint, model family, tokenizer, library versions, hardware, and whether the model was fine-tuned. If the task is supervised, the model is trained on labelled examples and tuned using a validation set. Hyperparameters such as learning rate, batch size, epoch count, random seed, and maximum sequence length should be recorded. If a pretrained model is used directly without fine-tuning, the paper must describe the prompting or inference method and its constraints. This draft does not name a specific model because the current project context does not confirm one.

D. Prediction outputs. For each processed record, a useful result structure may include a task name, predicted label, confidence or probability information when supported by the model, model identifier, model version, and processing timestamp. Confidence should not automatically be interpreted as a calibrated probability. Thresholds for high-priority review must be selected using validation data and operational requirements, and the interface should show when a result is uncertain.

E. Error analysis. Overall performance alone is insufficient. The final evaluation should inspect false positives and false negatives, confusion between related categories, performance on short or long messages, and errors associated with spelling variation or source-specific language. A model that performs well on formal portal text may perform differently on short social posts. Human reviewers should be able to correct errors and those corrections can inform later model updates, subject to appropriate governance.

F. Baseline and ablation. To support the claim that a transformer is useful, compare it with at least one simple baseline, such as a majority-class predictor or a conventional text-classification model. Where feasible, compare preprocessing variants or examine whether source metadata contributes to performance. The comparison must use the same test set and label definitions. Results should include the exact split and confidence intervals or repeated-run variability when feasible.

G. Human-in-the-loop use. Model outputs should be presented as suggestions for review, not final administrative decisions. Urgency predictions can influence which reports receive attention, so the policy must define what counts as urgent and how a human can override the score. A system should not automatically close, reject, or deprioritise a grievance solely because of a model prediction.

The transformer component is a central planned analytical capability, but a defensible research paper must distinguish a chosen architecture from a completed experiment. Until the exact checkpoint, task labels, and evaluation results are recorded, the paper should describe the method and evaluation plan rather than claim that the model achieved a particular accuracy.

## VII. DATABASE DESIGN AND BACKEND SERVICES

A relational database provides a consistent location for storing feedback records and derived analysis. The project uses MySQL for the database layer. The schema has been described as designed for citizen feedback and related analysis information. The final paper should include the actual table names, primary keys, foreign keys, constraints, and indexes from the implemented schema, rather than substitute a hypothetical design for the database that exists.

A. Data organisation. A typical design may store one row per feedback item and use a stable feedback identifier as the primary key. Analysis outputs can be stored in the same table for a simple prototype or in a separate table if multiple model versions or repeated analyses must be preserved. Separating raw feedback from predictions helps maintain provenance and allows predictions to be regenerated without overwriting the source record. Source names and categories should use consistent controlled values, and timestamps should have a documented timezone convention.

B. Indexing and query performance. The database should index fields commonly used in filters, such as source, timestamp, location, and category. Indexes should be chosen after examining actual queries because too many indexes increase storage and write costs. For large text fields, a full-text index may be useful for keyword exploration, depending on language and database configuration. Pagination is important so that the API does not return hundreds of thousands of rows in a single response.

C. FastAPI service. FastAPI is used as the backend framework in the project setup. It can expose endpoints that allow the dashboard to request feedback, filters, aggregate counts, and NLP results. Input validation should restrict pagination sizes and validate dates, identifiers, and category parameters. Responses should follow a consistent structure and return appropriate HTTP status codes. API documentation generated by the framework can support team integration and testing.

D. Integration status. The project context records that the FastAPI application was started with Uvicorn and that a health endpoint was checked. This demonstrates that the application can run and respond to a health request. It does not by itself prove that the database connection, data retrieval, prediction pipeline, or dashboard API calls are functioning. Those should be tested separately. In particular, a health endpoint should distinguish service liveness from dependency readiness if database availability is required for normal operation.

E. Security and configuration. Database credentials, secret keys, and environment-specific settings should be stored in environment variables or a protected configuration mechanism and excluded from source control. The backend should use parameterised database operations, validate incoming values, and apply authentication and authorisation where the deployment context requires it. Logs should not unnecessarily include full complaint text or personal details. CORS settings should be restricted to the expected dashboard origins in deployment rather than left broadly open by default.

F. Testing. Backend tests should cover a successful health response, database connection failure, retrieval of known records, empty result sets, invalid filters, pagination, malformed inputs, and error responses. Integration tests should verify that records returned by the API match database content. If analysis is asynchronous or computationally expensive, the service should manage timeouts and avoid blocking unrelated requests.

The database and API form the bridge between the data pipeline and dashboard. Their completion should be assessed through reproducible integration tests rather than inferred from the existence of a schema or a running development server.

## VIII. DASHBOARD AND USER WORKFLOW

The dashboard is designed to turn a large dataset into views that can be explored by a project reviewer or public-service analyst. The project screenshots show an overview page, feedback trends, a feedback explorer, and an AI/NLP insights section. The overview includes displayed values such as approximately 205,168 total feedback records, five data sources, 6,810 high-priority items, and 24 locations. These are dashboard display values reported in the prototype; they should not be presented as validated live system outputs until reconciled with the database and the definitions behind each metric.

A. Overview. The overview page can provide the number of records, source coverage, locations represented, and high-priority count. Each metric should have a clear definition and be generated from a consistent backend query. For example, “high-priority” should indicate whether it is based on a verified label, a human review, a model prediction, or a prototype rule. Counts should be updated when filters are applied and should make the time range visible.

B. Trend analysis. Trend charts can show the number of feedback records over time, potentially separated by source or category. The date field and aggregation interval must be consistent. Missing timestamps should be excluded or represented transparently. A rise in complaint volume does not necessarily prove that service quality has worsened; it may reflect changes in reporting behaviour, source coverage, or data collection.

C. Feedback explorer. A searchable explorer allows users to inspect individual messages, filter by source or location, and compare the original text with cleaned text and model outputs. Pagination and server-side filtering are important for performance. The interface should make source provenance visible and allow users to identify records with missing or low-confidence analysis.

D. AI/NLP insights. The prototype contains AI/NLP insight cards identified as prototype elements. Before describing them as operational results, each card should be connected to a documented algorithm or model output and validated against the stored records. If an insight is a manually created placeholder or static demonstration, the interface should label it as such. This avoids implying that an automated analysis has been executed when it has not.

E. Accessibility and interpretation. Charts should include clear labels, readable scales, and a text alternative or summary. Colours should not be the only way to communicate urgency or sentiment. Tooltips should explain metrics, and users should be able to view the underlying records that contribute to a summary. If a chart aggregates sensitive or small groups, the system should consider whether displaying the result could expose individual information.

F. Intended workflow. A user starts with the overview, narrows the time period or source in the trend view, opens the feedback explorer to inspect records, and then uses NLP outputs as additional context. Where a high-priority item is shown, the interface should expose the reason for the flag and provide access to the original message. The user remains responsible for deciding how to act on the information.

A useful dashboard is not merely a collection of attractive charts. It should be a transparent interface to data whose provenance, definitions, and analytical limitations are understood.

## IX. EVALUATION METHODOLOGY AND RESULTS STATUS

The available project context confirms development activity in data preprocessing, database setup, backend startup, and dashboard prototyping. It does not provide a verified transformer model name, a labelled test-set definition, or reproducible model evaluation metrics. Consequently, this paper does not report model accuracy, precision, recall, F1-score, or claims of improved performance. Those values should be added only after experiments have been run and their outputs saved.

A. Data pipeline evaluation. The preprocessing pipeline should be evaluated by recording the input record count, output record count, duplicate count, empty-text count, missing-value rates, and the number of records whose cleaned text differs from raw text. These metrics should be generated automatically from the same versioned dataset. A sample of cleaned messages should be reviewed manually to determine whether important meaning was accidentally removed.

B. Classification evaluation. For a labelled classification task, use a held-out test set that is not used for model selection. Report accuracy together with precision, recall, and F1-score. If class distribution is imbalanced, macro-F1 and per-class results are especially useful because a high accuracy may hide poor performance on a minority class. For urgent-case detection, false negatives may have greater operational cost than false positives, so threshold selection should reflect the documented use case.

C. Baseline comparison. The transformer model should be compared against a simple baseline using the same data splits and labels. The comparison should report not only metric differences but also inference cost, latency, and operational complexity. A more complex model is not automatically preferable if the improvement is small or if the latency and maintenance costs are unacceptable.

D. Source and time generalisation. Because feedback sources differ in style, the evaluation should report performance by source where the sample size permits. A time-based holdout can help estimate performance on newer messages. If location-level comparisons are intended, the study should test whether differences in language or reporting patterns cause unequal error rates. Small groups should not be over-interpreted.

E. Human review. A qualitative evaluation should examine a stratified sample of predictions, including correct predictions, errors, low-confidence outputs, and messages flagged as high priority. Reviewers should record the type of error and whether it could affect a decision. If multiple annotators are used, agreement should be reported and disagreements resolved under a documented protocol.

F. System evaluation. The software stack should be tested separately from the NLP model. Relevant measures include API response time, database query latency, dashboard loading time, throughput, failure behaviour, and successful end-to-end retrieval of feedback and associated analysis. Tests should use a representative volume of data rather than only a few hand-entered records.

G. Results currently supported. The current evidence supports describing the project dataset as approximately 205,168 records based on the reported project summary; describing preprocessing as implemented in prior work but requiring remeasurement for final publication; describing the MySQL schema and FastAPI setup as development work; and describing the dashboard as a prototype. It does not support claiming a validated classification model, measured model accuracy, improved grievance resolution, or deployment at scale. The distinction is important for academic integrity and should remain in the final version unless new experimental evidence is collected.

## X. ETHICAL, PRIVACY, AND OPERATIONAL CONSIDERATIONS

Citizen feedback may contain names, phone numbers, addresses, case identifiers, allegations, health details, or other sensitive information. The project should apply data minimisation and process only the fields needed for the stated analytical purpose. Access to raw records should be restricted to authorised users, and data should be retained only as long as justified by the project or applicable policy. Where data is shared for research or demonstration, personal identifiers should be removed or masked where appropriate.

Model predictions can introduce bias. Sources may represent different groups of people and different styles of writing. Formal complaints may contain structured language, while social-media messages may include slang, abbreviations, code-switching, or sarcasm. A model trained on one source can perform poorly on another. Performance should therefore be checked across relevant subsets, and limitations should be disclosed. If language coverage is limited, the system should not imply equal reliability for all languages.

Sentiment is not a measure of the legitimacy of a complaint. A politely worded report can describe a severe issue, while an angry message may describe a minor inconvenience. Similarly, urgency should be defined using service policy and contextual evidence rather than inferred solely from emotional tone. The dashboard should avoid using model outputs as the sole basis for denying, closing, or deprioritising requests.

Transparency is important for users who rely on the dashboard. The system should indicate when a label is model-generated, show the model version where practical, and provide a route to inspect the underlying message. It should support human correction and keep an audit trail for consequential changes. If a model is updated, predictions from different versions should not be silently mixed in a way that makes trends misleading.

Operationally, the system should handle model or database failures gracefully. If NLP inference is unavailable, feedback should still be retrievable where possible, and the dashboard should clearly mark analysis as unavailable rather than show stale predictions as current. Monitoring should detect failed jobs, unusual shifts in label distributions, and changes in input text. Model retraining should be governed by a documented process and evaluated before replacing a current version.

## XI. LIMITATIONS AND FUTURE WORK

Several limitations remain. First, the reported dataset and preprocessing figures must be regenerated from a versioned final dataset before publication. Second, the exact transformer architecture and task-specific setup have not been confirmed in the project information used for this draft. Third, a labelled test set and reproducible evaluation metrics are not available here. Fourth, dashboard values and AI/NLP cards need to be reconciled with live database and API outputs. Fifth, the status of complete backend-to-database-to-dashboard integration requires end-to-end testing.

Future work should proceed in a sequence that makes each improvement measurable. The team should freeze a dataset version and create a data dictionary; verify source counts and quality checks; define the NLP tasks and label taxonomy; select and document a model; create train, validation, and test partitions; and run baseline and transformer experiments. The team should then save evaluation outputs and representative error examples. Database migrations, API contracts, and dashboard queries should be tested against a shared environment so that team members can integrate their work reliably.

Additional research directions include multilingual and code-switched feedback, duplicate or near-duplicate complaint detection, topic discovery, explainable prioritisation, calibrated confidence, and privacy-preserving analytics. These should be added only when the core pipeline is reliable. A future user study could evaluate whether analysts can find relevant feedback faster or make more consistent decisions with the dashboard, but such benefits should be measured rather than assumed.

## XII. CONCLUSION

CIVICVOICE is a project to support the analysis of citizen feedback collected from multiple channels through a combined data, NLP, database, backend, and dashboard workflow. The project context reports a dataset of approximately 205,168 records, a preprocessing workflow, a MySQL database design, a FastAPI service setup, and a dashboard prototype with overview, trend, explorer, and AI/NLP views. These components provide the foundation for a common interface to large volumes of unstructured feedback.

Transformer-based NLP is a promising direction because contextual language representations can support more nuanced text analysis than keyword-only approaches. However, the usefulness of the model must be demonstrated through task-specific labels, baseline comparisons, held-out evaluation, and error analysis. Likewise, the dashboard should be described as a prototype until its displayed metrics and insights are confirmed to originate from tested backend and database operations.

The contribution of this paper is a transparent account of the system's motivation, architecture, current development status, and evaluation requirements. It deliberately avoids inventing a model checkpoint, accuracy score, or operational outcome. Completing the integration tests, documenting the final transformer configuration, measuring model performance, and addressing privacy and fairness will be essential next steps toward a reproducible and responsible citizen-feedback analysis system.

## REFERENCES

[1] A. Alkraiji, "Citizen satisfaction with government services: The role of digital service delivery," IEEE Access, 2020. DOI: 10.1109/ACCESS.2020.3004541. Verify the exact title, author metadata, volume, pages, and DOI against the publisher record before submission.

[2] Airlangga et al., work cited in the project materials, IEEE International Smart Cities Conference (ISC2), 2023. DOI: 10.1109/ISC257844.2023.10293572. Verify the complete author list and publication title against the official proceedings.

[3] Sarvestani, work cited in the project materials, IEEE Access, 2025. DOI: 10.1109/ACCESS.2025.3641669. Verify the title, author list, and publication details against the publisher record.

[4] CIVICVOICE Project Team, "AI Assistant for Multi-Channel Citizen Feedback Analysis," project repository. Available: https://github.com/CivicVoice-AI/AI-Assistant-for-Multi-Channel-Citizen-Feedback-Analysis. Accessed: 24 Oct. 2026.

[5] A final version should add verified foundational and task-specific references for transformer architectures, the exact pretrained model used, evaluation methodology, and responsible AI in public services. These references have not been fabricated here and should be inserted after the implementation choices are confirmed.